

## Atividade prática - Aula 04

Parte 2: O Desafio do Bug Oculto (Análise de Código)

1. Por que a linha `lista.forEach(...)` causará um erro fatal no console ( `TypeError: lista.forEach is not a function` )? Qual função faltou executar na variável `lista` ?

R: O `localStorage.getItem()` devolve o dado como `String`, e o `.forEach()` só funciona em `Arrays`.

2. Mesmo que a lista rodasse, por que as encomendas entregues voltariam a aparecer se o usuário der um F5 na página?

R: Porque as alterações foram feitas só na memória temporária. O código removeu os itens da lista local, mas esqueceu de salvar a nova lista no `localStorage` (`localStorage.setItem`). Quando recarrega, a página busca os dados antigos salvos.

Parte 3: Evidência de Execução (DevTools) **(NA OUTRA PÁGINA)**

# Painel de Controle de Entregas

Cadastrar Nova Encomenda

Código (ex: BR004)

Nome do Cliente

PENDENTE

Cadastrar Encomenda

| Código | Cliente    | Status      | Ações   |
|--------|------------|-------------|---------|
| GFM001 | Gustavo    | EM TRANSITO | Excluir |
| GFM002 | Tony Stark | ENTREGUE    | Excluir |
| GFM003 | Fulano     | PENDENTE    | Excluir |

Total: 3 Pendentes: 1 Em Trânsito: 1 Entregues: 1

Application

Manifest

Service workers

Storage

WebMCP NEW

Storage

Local storage

http://127.0.0.1:5500

Session storage

Extension storage

IndexedDB

Cookies

Private state tokens

Shared storage

Cache storage

Storage buckets

Background services

Back/forward cache

Background fetch

Background sync

Bounce tracking mitigati...

Notifications

Payment handler

Periodic background sync

Speculative loads

Push messaging

Reporting API

Device bound sessions

Frames

top

Filter

http://127.0.0.1:5500

| Key              | Value                                                        |
|------------------|--------------------------------------------------------------|
| banco_encomendas | [{"codigoRastreio":"GFM001","clienteNome":"Gustavo","stat... |

Payment handler

▼ [{"codigoRastreio": "GFM001", clienteNome: "Gustavo", status: "EM\_TRANSITO"},~]

▶ 0: {codigoRastreio: "GFM001", clienteNome: "Gustavo", status: "EM\_TRANSITO"}

▶ 1: {codigoRastreio: "GFM002", clienteNome: "Tony Stark", status: "ENTREGUE"}

▶ 2: {codigoRastreio: "GFM003", clienteNome: "Fulano", status: "PENDENTE"}

## Parte 4: Defesa Técnica Relâmpago (3 Perguntas Rápidas)

1. Ciclo de Conversão: Para que serve exatamente o comando `JSON.stringify` na hora de salvar e o `JSON.parse` na hora de ler os dados do `localStorage` ?

R: O `localStorage` só consegue armazenar texto (string). Ele não entende arrays, objetos ou números complexos.

2. Experiência do Usuário (UX): Por que bloqueamos o botão com `btnSubmit.disabled = true` antes de esperar o salvamento acontecer? O que aconteceria se um usuário impaciente clicasse 5 vezes seguidas rápido?

R: Bloqueamos o botão para prevenir que o mesmo item seja salvo múltiplas vezes no banco de dados.

3. Arquitetura: Se apagarmos a linha `await consultarEncomendas()` de dentro da função `cadastrarEncomenda()` , o dado é salvo? O que acontece com a tabela na tela imediatamente após o clique?

R: Sim, o dado é salvo normalmente. No entanto, a tabela na tela não será atualizada e continuará mostrando apenas as encomendas antigas, dando ao usuário a falsa sensação de que o cadastro falhou até que ele recarregue a página .